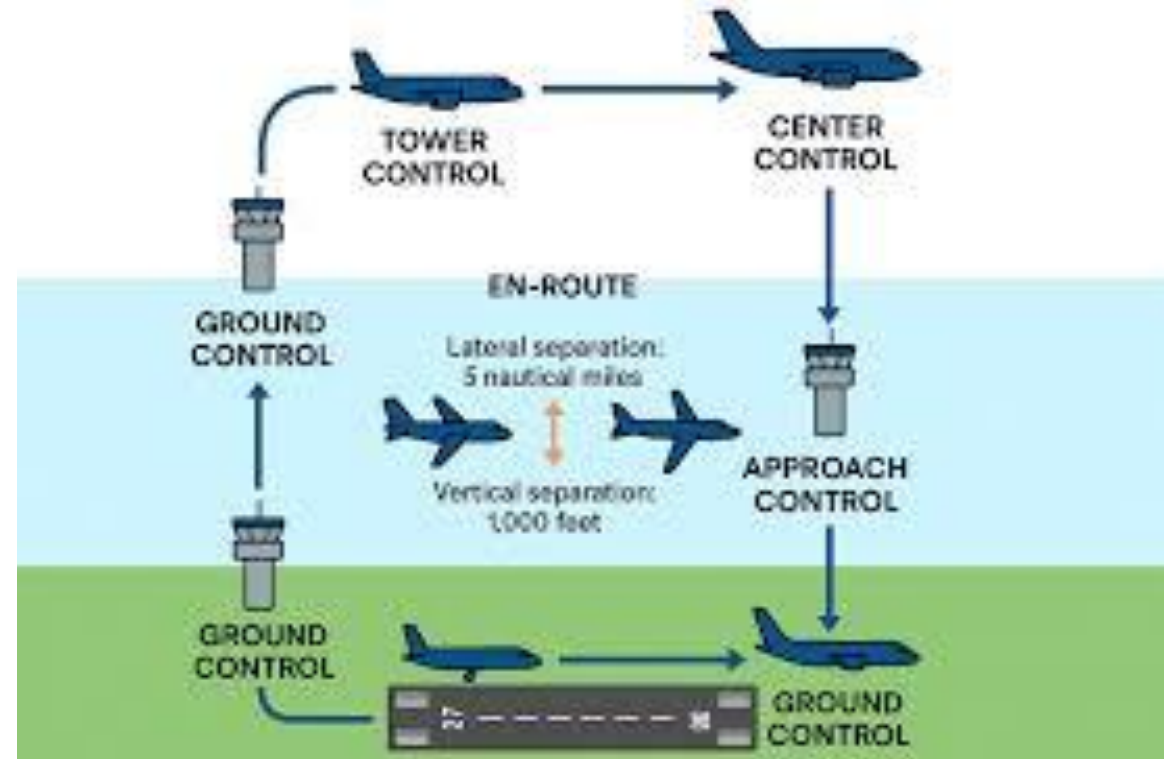


Welcome & Module Induction

- **What we learned previously:** In Module 12, we used Multi-threading and Multiprocessing to run tasks in parallel.
- **What we are learning today:** We are learning the "Next Generation" of concurrency: **Asynchronous Programming**. We will master the **Event Loop**, the **async/await** keywords, and the power of **Non-blocking I/O**.
- **Why this will be helpful:** In modern AI and Web frameworks (like FastAPI), asyncio is the secret to extreme speed. While a Threaded app might handle 100 users, an Asynchronous app can handle 10,000 on the same hardware. This is essential for building real-time AI agents and high-speed data scrapers.

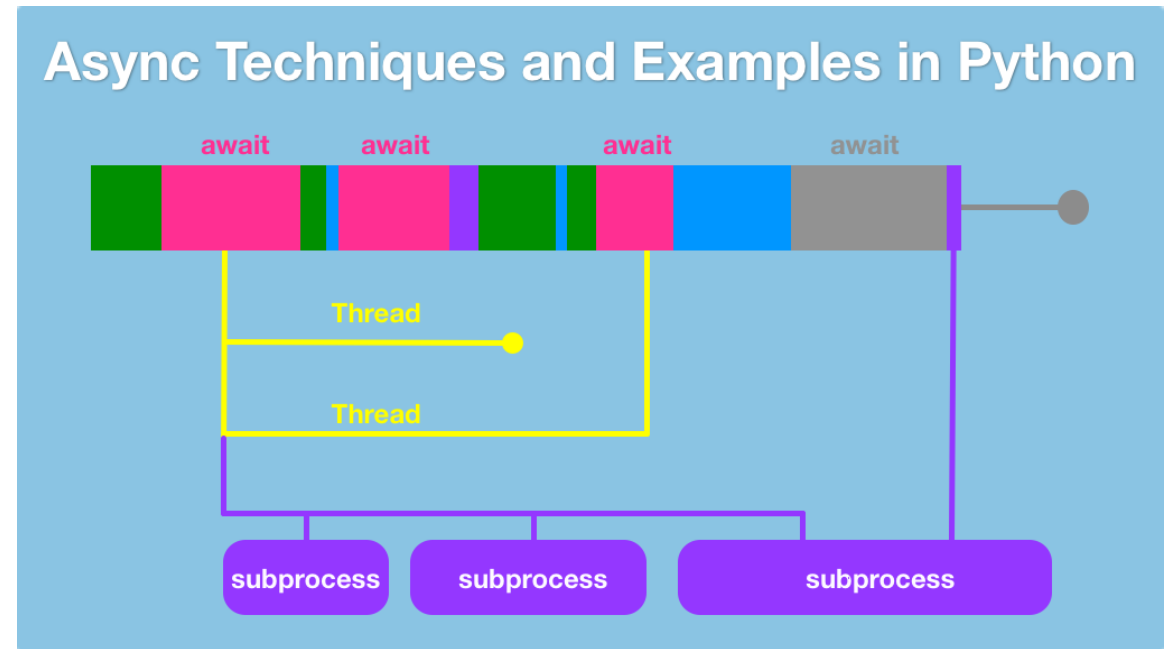
The Event Loop (The Air Traffic Controller)

- **The Problem:** In normal code, your script stops and waits for every task.
- **The Solution:** The Event Loop. It is a continuous loop that monitors tasks. If one task is waiting for the internet, the loop "pauses" it and immediately starts the next task.
- **The Goal:** Never let the CPU sit idle. Keep the "loop" spinning at all times.



async and await (The Modern Syntax)

- **async def:** This turns a normal function into a "Coroutine." It tells Python, "This function is allowed to pause itself."
- **await:** This is the pause button. It tells Python, "Go do other work until this specific line is finished."
- **The Clean Code Advantage:** Unlike older methods, async/await makes non-blocking code look and feel like a standard, readable script.



Non-blocking I/O (Fluid Systems)

- **Blocking:** Your app "freezes" while waiting for a database or a website. The user sees a spinning wheel.
- **Non-blocking:** Your app stays responsive. It can continue to process user clicks or other data while the "slow" database work happens in the background.

Figure 4. Typical flow of the asynchronous blocking I/O model (**select**)

